

537 notebook — operator manual

This note describes `537_Sports_Simulation.ipynb`, `sports/sim_config.py`, and how promotion, pools, and binning relate to each other. For implementation details, see `CELL 1` in the notebook (helper definitions).

Quick start

1. Edit `sports/sim_config.py` (population size, pools, seeds, defaults).
2. In Jupyter: run **Cell 0** (optional), **Cell 1** (helpers), **Cell 10** (widgets).
3. Cell 10 calls `reload_sim_config()` at the top; re-run it after saving `sim_config.py` to refresh globals and widget sync when using batch lock.

Batch diagnostic cells (2–9) were dropped from the live notebook on 2026-05-08. A freeze of the earlier version lives at `obsolete_files/sports_gameplan_old/537_Sports_Simulation_pre_archive_8May.ipynb`.

Core objects (per simulated person)

Symbol	Meaning
N (<code>N_INDIVIDUALS</code>)	People per run.
K (<code>N_WINNERS</code>)	Promotion slots per run ("draft picks", "tenure lines", etc.).
N_POOLS	Number of teams / pools. People are partitioned across pools (approximately equal roster sizes when using the equal-size template).
A or A_i	Latent ability for person <i>i</i> , drawn each run from the chosen ability law (A/B/C in config or widget). Clipped or scaled to [0, 1] depending on law.
pool_id	Integer team label (only when pools are on).
sorting_signal	Signal used only to assign pools: <code>A_i + ε_i</code> with <code>ε_i ~ Normal(0, SORTING_NOISE_SD^2)</code> if noise > 0; else equals A_i . True A_i is unchanged.
global_rank_score	Percentile rank of A within the run (larger = better).
local_rank_score	Percentile rank of A within (run , pool_id).
pool_quality	Pool mean of A (reported on each row; mean of A in that pool for that run).
promoted	Boolean: selected as a winner that run.

Ability distribution (`ABILITY_DISTRIBUTION_CHOICE` / widget)

Code	Widget label	Draw
A	Uniform(0,1)	<code>Uniform(0, 1)</code> .
B	clipped Normal(0.5, 0.18)	<code>Normal(0.5, 0.18)</code> clipped to [0, 1].
C	scaled Beta(2,5)	<code>Beta(2, 5)</code> then scaled by max so support matches notebook helper.

Promotion score modes (`score_mode` / widget "Promotion score")

Weights passed into **winner draw** are built from:

Mode	Weights (conceptually)
ability	<code>A_i</code> (not the main widget focus).
global_rank	Global rank score (no-pool reference world).
local_rank	Local rank score within pool (fixed internal mix of rank signal when this mode is selected from dropdown).
local_rank_plus_ability	<code>w * local_rank_score + (1 - w) * A_i</code> with slider <code>ADDITIVE w</code> (<code>ADDITIVE_LOCAL_RANK_WEIGHT</code> from config or widget).

Winner draw (`WINNER_SELECTION_CHOICE` / widget "Winner draw")

Code	Label	Mechanism
A	Weighted K without replacement	Normalized nonnegative weights → <code>numpy.random.Generator.choice(..., size=K, replace=False, p=p)</code> .
B	Bernoulli	Each person promoted independently with probability capped by <code>min(normalized_weight * K, 1)</code> (see notebook <code>choose_winners</code>).
C	Top-K	Deterministic: highest K weights win.

Pool assignment (`pool` / `LOCAL_POOL_ASSIGNMENT` patterns)

Uses `sorting_signal` (noisy or exact **A**):

Code	Pattern
A	Random equal-sized pools (permutation of a round-robin template).
B	Assortative: sort by signal, fill pools in order so similar signals share a pool.
C	Disassortative: sort by signal, assign to pools in modulo fashion to mix highs and lows.

Caption under plots switches to “noisy assortative / disassortative” when `SORTING_NOISE_SD > 0`.

Cell 10: binning modes (how the x-axis is built)

A) Individuals: `pd.qcut` on x

- **Person bins** slider (or `N_BINS` when batch lock): number of **equal-count** quantile bins over **all person-rows** in the simulation output, on the chosen x column (`local_rank_score` , `global_rank_score` , or `A`).
- Plot: line (`plot_bin_summary`): $x \approx$ bin mean of x, $y =$ mean `promoted` in bin.

B) Pools: equal pool count per bin (`method="equal_count"`)

- **Pool–talent bins K** (or `N_POOL_AGG_BINS` with batch lock).
- **Within each run:** compute each pool’s mean **A**; **sort pools** by that mean; split pool IDs into **K** contiguous blocks with `numpy.array_split` (as close to equal **number of pools per bin** as possible).
- **Constraint:** need `N_POOLS ≥ K` .
- **Across runs:** average bin-level mean **A** and promotion rate (see `summarize_by_pool_mean_A_bins`).
- Plot: bars Q1...QK (`plot_pool_A_bin_summary`) with Q1 = lowest mean-talent bin.

C) Pools: equal width on mean A (`method="equal_width"`)

- Same **K**, but **within each run** build **K** equal-width intervals from **min** to **max** of **pool mean A** (`pd.cut`).
- **Some intervals may contain zero pools** in a given run; those bins vanish or carry no mass for that run—fewer than **K** bars can appear after averaging.
- Use when you want bins comparable to **equal-width** talent bands rather than equal team counts.

Config vs widget: who wins?

Knob	Primary location	Notes
N, K, N_POOLS, seeds, N_BINS, N_POOL_AGG_BINS, noise default, w default	<code>sim_config.py</code>	Reload via Cell 0 / Cell 10 start or <code>reload_sim_config()</code> .
Batch lock	Cell 10 checkbox	When on: N , runs, both bin counts track <code>N_INDIVIDUALS</code> , <code>N_RUNS</code> , <code>N_BINS</code> , <code>N_POOL_AGG_BINS</code> .
Ability, winner draw, pool mode, promotion score, w, noise, plot view, binning mode	Cell 10 widgets	Override config for interactive EDA without editing file (except N_POOLS , K still from config unless you change file).
Figure size	<code>FIG_PLAYGROUND_EDA_INCHES</code> in <code>sim_config.py</code>	Inches (width, height) for the single EDA figure.

File map

Path	Role
<code>sports/sim_config.py</code>	<code>RUN_CELL1</code> , <code>RUN_CELL_PLAYGROUND</code> , N , K , N_POOLS , N_BINS , N_POOL_AGG_BINS , seeds, figure size, default A/B/C choices, sorting noise default, additive w default.
<code>sports/537_Sports_Simulation.ipynb</code>	Cell 1 : helpers (<code>simulate_population_rows</code> , binning, plotting). Cell 10 : widgets.
<code>obsolete_files/sports_gameplan_old/537_Sports_Simulation_pre_archive_8MAy.ipynb</code>	Snapshot before batch cells 2–9 were removed from the live notebook (2026-05-08).
<code>sports/documents/537_Manual.md</code>	This document.

Troubleshooting

Symptom	Likely cause
<code>NameError</code> for helpers	Run Cell 1 (<code>RUN_CELL1 = True</code>).
Pool binning + no pools	Choose a pooled plot view or switch binning to individuals.
"N_POOLS must be $\geq K$ "	Equal-count pool binning only; lower K or raise N_POOLS in <code>sim_config.py</code> .
Fewer than K pool bars (equal width)	Some width bins empty in some runs—normal for equal-width on sparse pool means.

Version note

- Cell 10 no longer ships a **Cell 8a preset** (explicit sliders and `sim_config` only).
- On **2026-05-08** the live notebook dropped **CELL 2–9**; the frozen copy lives at `obsolete_files/sports_gameplan_old/537_Sports_Simulation_pre_archive_8MAy.ipynb` , and `sim_config.py` no longer exposes `RUN_CELL2` ... `RUN_CELL9` .